

BidKar

The Premier Auction Platform

Frontend API Reference Documentation

Version 2.0 | June 2026

Base URL (Development): *http://localhost:3000*

1. Overview

BidKar is a real-time auction platform backend. This document covers all API endpoints a frontend developer needs to integrate, including authentication, item management, payments, wallet, transactions, and auction settlement.

1.1 Global Conventions

Property	Value
Base URL (dev)	http://localhost:3000
Content-Type (JSON)	application/json
Content-Type (upload)	multipart/form-data
Auth header	Authorization: Bearer
Price unit	Paise (divide by 100 for INR display)
OTP validity	2 minutes
Max photos per item	5 files (field name: "photos")

1.2 User Roles

Every authenticated user has a role that controls access:

Role	Permissions
USER	Browse items, bid, manage wallet, view transactions
SELLER	All USER permissions + create/list auction items
ADMIN	All SELLER permissions + administrative actions

1.3 Item Status Lifecycle

Item statuses are managed automatically by background cron jobs:

DRAFT → ACTIVE → SOLD

- **DRAFT:** Item is scheduled, not yet live. AuctionStarter cron activates it at startTime.
- **ACTIVE:** Item is live and accepting bids. AuctionCloser cron closes it at endTime.
- **SOLD:** Auction is complete. Settlement has been processed and invoice is available.

NOTE Frontend should poll item status periodically (or use WebSockets when available) to keep auction state up to date. Background crons run every ~60 seconds.

2. Authentication

All auth routes live under `/api/auth`. On successful login or OTP verification, the server returns a JWT. Send this token in the Authorization header for all protected routes.

2.1 Endpoints

Method	Endpoint	Description	Auth
POST	<code>/api/auth/register</code>	Send signup OTP to email	Public
POST	<code>/api/auth/verify</code>	Verify OTP — returns JWT token	Public
POST	<code>/api/auth/login</code>	Login with email + password	Public
POST	<code>/api/auth/logout</code>	Logout current session	Public
GET	<code>/api/auth/profile</code>	Get current user profile	Bearer token
POST	<code>/api/auth/forgot-password</code>	Request password reset OTP	Public
POST	<code>/api/auth/reset-password</code>	Reset password with OTP	Public

2.2 Register — POST `/api/auth/register`

Initiates the signup flow by sending an OTP to the provided email.

Request body:

Field	Type	Notes
username	string	Chosen display name
email	string	Valid email address — OTP will be sent here
password	string	Must be strong (e.g. Krish@123)
role	string	"USER" for buyers, "SELLER" for sellers

Sample request:

```
{ "username": "krish", "email": "user@example.com", "password": "Krish@123", "role": "USER" }
```

Response 200:

```
{ "success": true, "message": "OTP sent successfully to your email" }
```

UX TIP

OTP is valid for 2 minutes. Show a countdown timer and offer a "Resend OTP" button after expiry.

2.3 Verify OTP — POST /api/auth/verify

Verifies the OTP and returns a JWT token plus user info.

Request body:

Field	Type	Notes
email	string	Same email used during register
otp	string	6-digit OTP received by email

Response 200:

```
{ "success": true, "token": "", "user": { "userId": "...", "username": "krish",  
  "email": "...", "role": "USER" } }
```

IMPORTANT

Store the token securely (memory / secure cookie). The "role" field determines which UI features to show.

2.4 Login — POST /api/auth/login

Request body:

Field	Type	Notes
email	string	Registered email
password	string	Account password

Response 200:

```
{ "success": true, "token": "", "user": { "userId": "...", "username": "...",  
  "email": "...", "role": "USER|SELLER|ADMIN" } }
```

2.5 Forgot Password — POST /api/auth/forgot-password

Request body:

```
{ "email": "user@example.com" }
```

Response 200:

```
{ "success": true, "message": "Password reset OTP sent successfully to your email" }
```

2.6 Reset Password — POST /api/auth/reset-password

Request body:

Field	Type	Notes
email	string	Account email
otp	string	6-digit reset OTP
newPassword	string	New password (strong)

Response 200:

```
{ "success": true, "message": "Password reset successful. You can now login with your new password" }
```

3. Items

Item routes expose a standard REST interface for auction items. Public routes do not require a token. The upload route requires SELLER or ADMIN role.

3.1 Endpoints

Method	Endpoint	Description	Auth
GET	/api/items	Get all active auction items	Public
GET	/api/items/:id	Get a single item by ID	Public
POST	/api/items	Create / upload a new item	Bearer token [SELLER/ADMIN]

3.2 Get All Items — GET /api/items

Returns all items with ACTIVE status. Results are Redis-cached — first request hits the database (~225ms), subsequent requests are served from cache (~9ms).

Response 200:

```
{ "success": true, "source": "database|cache", "count": 1, "items": [ { ...item } ] }
```

REDIS The "source" field will be "database" or "cache". You can use this to show a subtle "live" vs "cached" indicator in the UI if desired.

3.3 Get Item by ID — GET /api/items/:id

Response 200:

```
{ "success": true, "source": "database", "item": { "_id": "...", "title": "Vintage Rolex",  
  "description": "Mint condition watch from 1980", "startingPrice": 200000,  
  "currentHighestBid": 200000, "photos": ["https://res.cloudinary.com/..."],  
  "sellerId": "...", "winnerId": null, "status": "ACTIVE",  
  "endTime": "2026-12-31T23:59:00.000Z", "startTime": "..." } }
```

Item response fields:

Field	Type	Notes
_id	string	MongoDB ObjectId — use as item identifier
title	string	Auction item name
description	string	Item description
startingPrice	number	In paise — divide by 100 for INR

Field	Type	Notes
currentHighestBid	number	In paise — current winning bid
photos	string[]	Cloudinary URLs — use directly in img src
status	string	DRAFT ACTIVE SOLD
sellerId	string	MongoDB ID of the seller
winnerId	string null	Null until auction ends
startTime	ISO date	When auction went/goes live
endTime	ISO date	When auction closes

3.4 Create Item — POST /api/items

Requires SELLER or ADMIN role. Must be sent as multipart/form-data (not JSON) because photos are file uploads.

Request — multipart/form-data:

Field	Type	Notes
title	string	Item name
description	string	Item description
startingPrice	number	Starting bid in paise
endTime	ISO date	Auction end datetime (e.g. 2026-12-31T23:59:00Z)
photos	File[]	Image files — field name MUST be "photos", max 5 files

Response 201:

```
{ "success": true, "message": "Auction item listed successfully", "item": { ...full item object } }
```

WARNING A 403 error is returned if the user role is not SELLER or ADMIN. Disable or hide the upload button for USER role accounts.

4. Payments

BidKar uses Razorpay for wallet top-ups. The flow is: frontend requests an order ID → opens Razorpay checkout → Razorpay silently calls the webhook → wallet is credited.

4.1 Endpoints

Method	Endpoint	Description	Auth
POST	/api/payments/create-order	Create Razorpay order for top-up	Bearer token
POST	/api/payments/webhook	Razorpay webhook (server use only)	Public

4.2 Create Order — POST /api/payments/create-order

Call this to initiate a wallet top-up. The returned orderId is passed to the Razorpay JS SDK to open the payment popup.

Request body:

Field	Type	Notes
coinsRequested	number	Amount of coins to add (e.g. 2.5)

Response 200:

```
{ "success": true, "orderId": "order_StAoGB3dEk2EjF", "amount": 250, "currency": "INR" }
```

Razorpay integration flow:

- Call /create-order and get the orderId.
- Open Razorpay checkout popup using the Razorpay JS SDK with the orderId.
- After user pays, Razorpay silently calls /api/payments/webhook on the server.
- Poll GET /api/wallet/balance to confirm that the wallet has been credited.

NOTE The webhook route is called by Razorpay in the background — it is not called by the frontend. A live internet URL is required for Razorpay to reach the webhook in production.

5. Wallet

Each user has a wallet for bidding. Balances are in INR. A wallet can be ACTIVE or SUSPENDED. Suspended wallets cannot place bids.

5.1 Endpoints

Method	Endpoint	Description	Auth
GET	/api/wallet/balance	Get current wallet balance	Bearer token

5.2 Get Balance — GET /api/wallet/balance

Response 200:

```
{ "success": true, "data": { "availableMoney": 2.5, "frozenMoney": 0, "currency": "INR" } }
```

Wallet fields:

Field	Type	Notes
availableMoney	number	Funds available to use for bidding
frozenMoney	number	Funds locked against active bids
currency	string	Always "INR"
status	string	"ACTIVE" or "SUSPENDED" — check before allowing bids

WARNING If wallet status is "SUSPENDED", block all bidding UI and show a suspension warning. Suspension happens automatically if buyer does not complete offline payment within 48 hours.

6. Transactions

Transaction history tracks all wallet top-ups for a user. Results are paginated.

6.1 Endpoints

Method	Endpoint	Description	Auth
GET	/api/transaction/history	Get paginated transaction history	Bearer token

6.2 Transaction History — GET /api/transaction/history

Response 200:

```
{ "success": true, "count": 1, "total": 1, "totalPages": 1, "currentPage": 1,
  "data": [ { "_id": "...", "razorpayOrderId": "order_...", "razorpayPaymentId": "pay_...",
    "amountInPaise": 250, "coinsToBeAdded": 2.5, "status": "SUCCESS",
    "createdAt": "2026-05-24T10:46:39.473Z" } ] }
```

Transaction status values:

Status	Meaning
PENDING	Order created, payment not yet confirmed by Razorpay
SUCCESS	Payment confirmed and wallet credited

Use totalPages to render pagination controls in the transaction history UI.

7. Auction Settlement

Settlement is triggered automatically by the AuctionCloser cron after an auction ends. It calculates hammer price, collects 10% security deposit from the winner's wallet, and generates a downloadable invoice.

7.1 Endpoints

Method	Endpoint	Description	Auth
POST	/api/settlements/:itemId/settle	Trigger settlement for an item	Public
GET	/api/settlements/:settlementId/invoice	Download PDF invoice	Bearer token

7.2 Settlement Response Fields

Field	Type	Notes
settlementId	string	Unique ID (e.g. SETTLE-319721-689) — needed to download invoice
hammerPriceRupees	string	Final winning bid amount in INR
securityDepositRupees	string	10% of hammer price — paid via BidKar wallet
offlineBalanceRupees	string	90% of hammer price — paid directly to seller
status	string	"COMPLETED" "PENDING" "DISPUTED"

7.3 Download Invoice — GET /api/settlements/:settlementId/invoice

Returns a PDF binary stream. Trigger the download using one of these approaches:

- Use an anchor tag with a download attribute and the full URL.
- Or fetch the response as a blob and use `URL.createObjectURL()` to trigger download.

NOTE Invoice is only downloadable when settlement status is "COMPLETED". The invoice shows buyer name, seller name, hammer price, security deposit (10%), and offline balance due (90%).

8. Background Workers (Cron Jobs)

The server runs 5 automatic background workers. Frontend does not call these directly, but their effects must be handled in the UI.

Worker	Trigger	Frontend Impact
AuctionStarter	Runs every ~60s. Activates DRAFT items where startTime has passed.	Poll item status. Show "Starting soon" countdown.
AuctionCloser	Runs every ~60s. Closes ACTIVE items where endTime has passed.	Poll item status. Trigger settlement flow UI.
settlementMonitor	Watches PENDING settlements. Triggers dispute after 48hrs.	Show pending settlement warning to buyer.
cleanupTransaction	Removes stale PENDING transactions.	No direct UI change needed.
cleanupUsers	Removes unverified accounts older than X hours.	No direct UI change needed.

8.1 Dispute / Deadbet Logic

If a buyer wins an auction but does not confirm offline payment within 48 hours, the settlementMonitor worker automatically:

- Changes settlement status from "PENDING" to "DISPUTED".
- Changes the buyer's wallet status to "SUSPENDED".

UI ACTION Check settlement status on the buyer dashboard. If "DISPUTED", show a prominent warning and block further bidding until resolved.

9. Google OAuth

Both buyer and seller can log in with their Google account. The backend handles the entire OAuth handshake — the frontend only needs to trigger a redirect and catch the JWT token on the return page.

9.1 Endpoints

Method	Endpoint	Auth	Description
GET	/api/auth/google	Public	Redirects to Google consent screen
GET	/api/auth/google/callback	Public	Google calls this — do NOT call manually
GET	/api/auth/google/failure	Public	Returns 401 if Google auth fails

9.2 Step 1 — Trigger Login

Redirect the browser to the backend URL. Use a button or anchor — NOT fetch/axios.

```
window.location.href = "http://localhost:3000/api/auth/google";

window.location.href =
"http://localhost:3000/api/auth/google">
Continue with Google
```

9.3 Step 2 — Handle the Callback Page

After Google login, the backend redirects to:

```
http://localhost:5173/auth/google/success?token=
```

Create a route at /auth/google/success that reads the token from the URL:

```
const params = new URLSearchParams(window.location.search);
const token = params.get("token");
if (token) {
  localStorage.setItem("token", token);
  window.location.href = "/dashboard";
}
```

9.4 Step 3 — Use the Token

The JWT works exactly like a regular login token — same Authorization header:

```
axios.get("/api/auth/profile", {
  headers: { Authorization: `Bearer ${localStorage.getItem("token")}` }
});
```

9.5 Failure Handling

If Google auth fails, the backend redirects to: http://localhost:5173/auth?error=google_failed

```
const params = new URLSearchParams(window.location.search);  
if (params.get("error") === "google_failed") { /* show error */ }
```

9.6 Notes

- Google OAuth users are auto-verified (isVerified: true) — no OTP step needed.
- If a user already has a local account with the same email, accounts are automatically linked.
- JWT payload is identical to regular login: { userId, role }.
- Token expiry: 1 day.

10. Filters & Search

Two public endpoints — no auth required. All filter params are optional URL query parameters.

10.1 Endpoints

Method	Endpoint	Auth	Description
GET	/api/items/filter-options	Public	Get dropdown values for filter UI
GET	/api/items	Public	Get items with filters, search, sort, pagination

10.2 Get Filter Options — GET /api/items/filter-options

Call once on page load to populate all filter dropdowns dynamically.

Response:

```
{ "success": true,
  "categories": ["Electronics", "Art", "Vehicles", "Fashion", "Furniture",
    "Collectibles", "Jewellery", "Books", "Sports", "Other"],
  "conditions": ["NEW", "LIKE_NEW", "GOOD", "FAIR"],
  "sortOptions": [
    { "value": "newest", "label": "Newest First" },
    { "value": "endTime_asc", "label": "Ending Soon" },
    { "value": "endTime_desc", "label": "Ending Last" },
    { "value": "price_asc", "label": "Price: Low to High" },
    { "value": "price_desc", "label": "Price: High to Low" }
  ] }
```

10.3 Get Items — GET /api/items

All params are optional. Append to URL as query string.

Param	Type	Default	Description
page	number	1	Page number
limit	number	12	Items per page (max 50)
category	string	—	Exact match (e.g. Electronics)
condition	string	—	NEW / LIKE_NEW / GOOD / FAIR
minPrice	number	—	Min currentHighestBid in INR
maxPrice	number	—	Max currentHighestBid in INR
search	string	—	Full-text search on title + description
sort	string	newest	Use values from filter-options
endingSoon	boolean	—	true = ending within 60 minutes

Param	Type	Default	Description
location	string	—	Partial city match (case-insensitive)
status	string	ACTIVE	ACTIVE / SOLD / CANCELLED
sellerId	string	—	All items from one seller

10.4 Condition Values

Value	Meaning
NEW	Sealed / never used
LIKE_NEW	Used once or twice, no visible wear
GOOD	Normal use, minor signs of wear
FAIR	Heavy use, visible wear but functional

10.5 Example Requests

```
// Basic browse
GET /api/items?page=1&limit;=12

// Category + price filter
GET /api/items?category=Electronics&minPrice;=500&maxPrice;=50000

// Search
GET /api/items?search=vintage+rolex

// Ending soon
GET /api/items?endingSoon=true&sort;=endTime_asc
```

10.6 Axios Example

```
const res = await axios.get("http://localhost:3000/api/items", {
  params: {
    page: currentPage,
    limit: 12,
    category: selectedCategory,
    minPrice: priceRange[0],
    maxPrice: priceRange[1],
    sort: selectedSort,
    search: searchQuery
  }
});
```

10.7 Response Shape


```
{ "success": true, "source": "database", // or "cache"  
  "count": 12, "total": 47, "totalPages": 4, "currentPage": 1,  
  "items": [ { ...item } ] }
```

NOTE Results are Redis-cached per unique filter combination (60s TTL). Use totalPages to render pagination controls.

11. Reviews

Both buyer and seller can leave a review after a settlement is COMPLETED. One review per person per transaction — enforced at database level.

11.1 Endpoints

Method	Endpoint	Auth	Description
GET	/api/reviews/user/:userId	Public	All reviews about a user + avg rating
POST	/api/reviews	Bearer token	Submit a review
GET	/api/reviews/mine	Bearer token	Reviews I have written
PATCH	/api/reviews/:id/report	Bearer token	Flag a review as troll
DELETE	/api/reviews/:id	Bearer token (ADMIN)	Soft-delete a review

11.2 Get Reviews for a User — GET /api/reviews/user/:userId

Public. Use this for seller profiles and buyer profiles. Supports ?page=1&limit=10

Response:

```
{ "success": true,
  "stats": { "avgRating": 4.5, "totalCount": 18,
    "breakdown": { "5": 10, "4": 5, "3": 2, "2": 1, "1": 0 } },
  "pagination": { "currentPage": 1, "totalPages": 2, "total": 18 },
  "reviews": [ { "rating": 5, "comment": "Great seller.",
    "tags": ["ACCURATE_DESCRIPTION", "PUNCTUAL"],
    "reviewerRole": "BUYER", "reviewerId": { "username": "preet", "avatar": null },
    "auctionId": { "title": "Vintage Rolex" },
    "createdAt": "2026-06-20T10:00:00.000Z" } ] }
```

11.3 Submit a Review — POST /api/reviews

Requires Bearer token. Settlement must be COMPLETED.

Request body:

Field	Type	Required	Notes
settlementId	string	Yes	MongoDB ID of AuctionSettlement
rating	number	Yes	1 to 5
comment	string	No	Max 500 characters
tags	string[]	No	See allowed tags below

11.4 Allowed Tags by Role

BUYER reviewing seller:

ACCURATE_DESCRIPTION, PUNCTUAL, GOOD_COMMUNICATION, SAFE_HANDOFF, ITEM_AS_EXPECTED

SELLER reviewing buyer:

ON_TIME, GOOD_COMMUNICATION, PAID_PROMPTLY, TRUSTWORTHY

11.5 Error Cases

Status Code	Reason
403	Settlement not COMPLETED yet
403	User not buyer or seller of this auction
409	Already submitted a review for this transaction
400	Used wrong tags for your role

11.6 UI Notes

- Show stats.avgRating as a star display on profile cards.
- Show stats.breakdown as a bar chart (10 people gave 5 stars).
- Only show "Write a Review" button when settlement.status === "COMPLETED".
- Check reviewerRole to show "Buyer review" vs "Seller review" label.

12. KYC (Identity Verification)

Verifies a user identity via Aadhaar OTP. Required before placing bids. All routes require authentication.

12.1 Endpoints

Method	Endpoint	Auth	Description
POST	/api/kyc/initiate	Bearer token	Start KYC — send OTP to Aadhaar-linked mobile
POST	/api/kyc/verify-otp	Bearer token	Submit OTP to complete verification
GET	/api/kyc/status	Bearer token	Get current KYC status
GET	/api/kyc/audit-logs	Bearer token (ADMIN)	View KYC audit logs

12.2 KYC Status Values

Status	Meaning
Unverified	User has not started KYC
Pending	OTP sent, awaiting verification
Verified	KYC complete — user can bid
Failed	Verification failed

12.3 Get Status — GET /api/kyc/status

Check on page load to know what state the user is in.

```
{ "success": true, "kycStatus": "Unverified", "userId": "..." }
```

12.4 Initiate Verification — POST /api/kyc/initiate

Submit Aadhaar number to trigger OTP to the user's Aadhaar-linked mobile.

```
{ "aadhaarNumber": "123456789012" }
```

Response:

```
{ "success": true,  
  "message": "OTP sent to your Aadhaar-linked mobile number",  
  "verificationRequestId": "abc123..." }
```

NOTE Save verificationRequestId — you need it in the next step.

12.5 Verify OTP — POST /api/kyc/verify-otp

```
{ "verificationRequestId": "abc123...",  
  "otp": "123456" }
```

Response 200:

```
{ "success": true, "message": "KYC verified successfully", "kycStatus": "Verified" }
```

12.6 OTP Error Responses

Code	Message	Extra Field
INVALID_OTP	OTP is incorrect	attemptsRemaining: 2
OTP_LOCKED	Too many attempts	unlocksAt: "2026-06-20T11:30:00Z"
REQUEST_EXPIRED	Session expired — restart from /initiate	—
RATE_LIMITED	Too many attempts	retryAfter: seconds

12.7 Admin Audit Logs — GET /api/kyc/audit-logs

Query params: userId, action, startDate, endDate, page, limit

Action types:

KYC_INITIATED | OTP_SUBMITTED | OTP_VERIFIED | VERIFICATION_FAILED | VERIFICATION_LOCKED

12.8 Recommended UI Flow

1. GET /api/kyc/status on page load

- Unverified → show Aadhaar input form
- Pending → show OTP input
- Verified → show green badge
- Failed → show retry option

2. User enters Aadhaar → POST /api/kyc/initiate

- Save verificationRequestId in component state

3. User enters OTP → POST /api/kyc/verify-otp

- Success → update UI to Verified
- Error → show attemptsRemaining or unlocksAt countdown

12.9 Notes

- Raw Aadhaar number is never stored — only a hashed reference.
- OTP goes to mobile linked to Aadhaar card, not the user email.
- KYC is required before a user can place any bids.
- Verification sessions expire automatically — user must restart if too slow.

13. Error Handling

All error responses follow the same structure. Always check the "success" field first.

HTTP Code	Meaning	Common Cause
200	Success	Request completed successfully
201	Created	Resource created (e.g. new item uploaded)
401	Unauthorized	Missing or expired JWT token
403	Forbidden	Valid token but insufficient role
404	Not Found	Item or resource ID does not exist
400	Bad Request	Missing required fields or invalid data

Standard error response shape:

```
{ "success": false, "message": "Access Denied: Only verified sellers can list auction items." }
```

14. Quick Reference — All Endpoints

Method	Endpoint	Description	Auth
POST	/api/auth/register	Send signup OTP	Public
POST	/api/auth/verify	Verify OTP → JWT	Public
POST	/api/auth/login	Login	Public
POST	/api/auth/logout	Logout	Public
GET	/api/auth/profile	Get profile	Bearer token
POST	/api/auth/forgot-password	Request reset OTP	Public
POST	/api/auth/reset-password	Reset password	Public
GET	/api/items/filter-options	Get filter dropdown values	Public
GET	/api/items	All active items (filtered/cached)	Public
GET	/api/items/:id	Single item	Public
POST	/api/items	Create item	Bearer token SELLER+
POST	/api/payments/create-order	Create Razorpay order	Bearer token
POST	/api/payments/webhook	Razorpay webhook (server)	Public
GET	/api/wallet/balance	Wallet balance	Bearer token
GET	/api/transaction/history	Transaction history	Bearer token
POST	/api/settlements/:itemId/settle	Process settlement	Public
GET	/api/settlements/:settlementId/invoice	Download invoice PDF	Bearer token
GET	/api/reviews/user/:userId	Get user reviews	Public
POST	/api/reviews	Submit review	Bearer token
GET	/api/reviews/mine	My written reviews	Bearer token
PATCH	/api/reviews/:id/report	Report review	Bearer token
DELETE	/api/reviews/:id	Delete review	Bearer token (ADMIN)
POST	/api/kyc/initiate	Start KYC	Bearer token
POST	/api/kyc/verify-otp	Verify KYC OTP	Bearer token
GET	/api/kyc/status	KYC status	Bearer token
GET	/api/kyc/audit-logs	KYC audit logs	Bearer token (ADMIN)

End of Document

BidKar Auction Platform • support@bidkar.com • www.bidkar.com